THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 7

Aadhya Maheshwari—1024150212

1. Swap two variables of any data type

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

template <class T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;

    cout << "Before swap: " << x << " " << y << endl;
    swapValues(x, y);
    cout << "After swap: " << x << " " << y << endl;

    return 0;
}
```

```
Before swap: 10 20
After swap: 20 10


=== Code Execution Successful ===
```

---

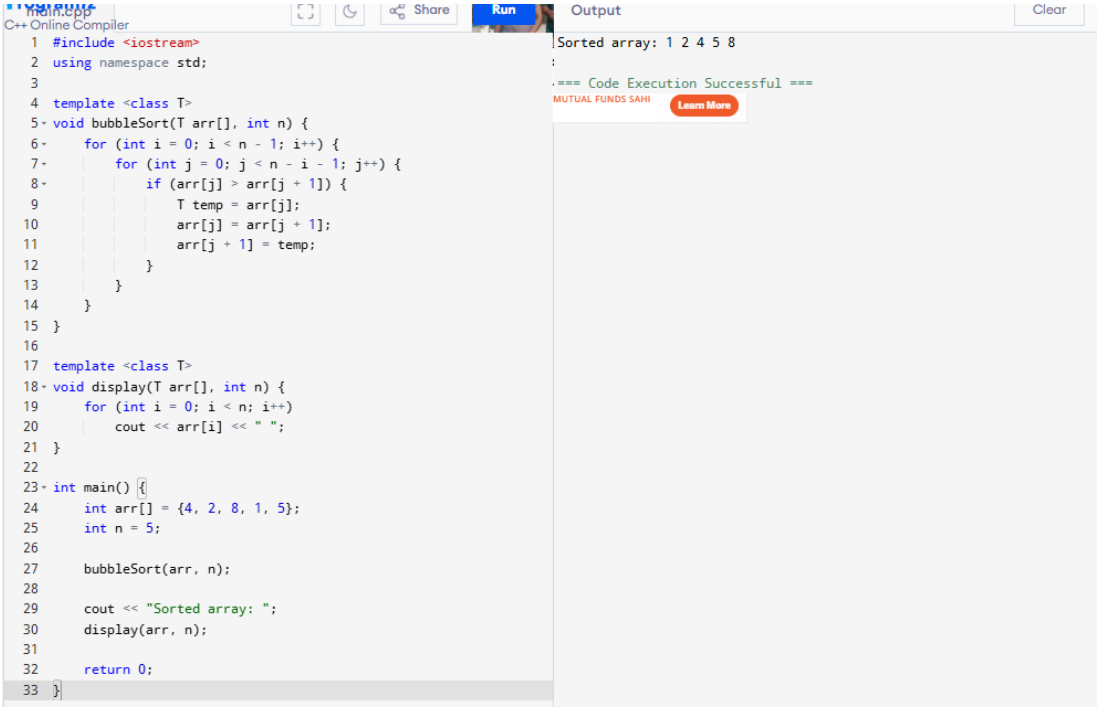
2. Find minimum element in an array

main.cpp | Share | Run | Output

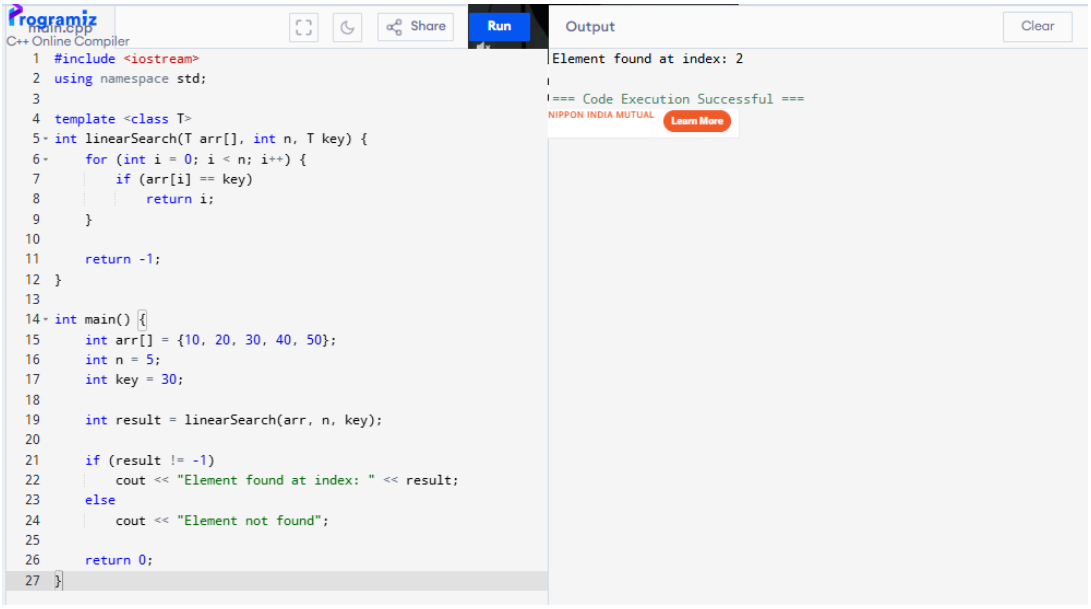
```cpp
#include <iostream>
using namespace std;

template <class T>
T findMin(T arr[], int n) {
    T minVal = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] < minVal)
            minVal = arr[i];
    }

    return minVal;
}

int main() {
    int arr[] = {5, 2, 9, 1, 7};
    int n = 5;

    cout << "Minimum element: " << findMin(arr, n);

    return 0;
}
```

```
Minimum element: 1

=== Code Execution Successful ===
```

3. Sort an array using bubble sort



4. Perform linear search



5. Overloaded function template process()

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;

template <class T>
void process(T a) {
    cout << "Single parameter: " << a << endl;
}

template <class T>
void process(T a, T b) {
    cout << "Two parameters of same type: " << a << " " << b <<
        endl;
}

template <class T, class U>
void process(T a, U b) {
    cout << "Two parameters of different types: " << a << " " <<
        b << endl;
}

int main() {
    process(10);
    process(10, 20);
    process(10, 5.5);

    return 0;
}
```

Output Clear

```
Single parameter: 10
Two parameters of same type: 10 20
Two parameters of different types: 10 5.5


=== Code Execution Successful ===
```

Class Templates

1. Stack with push and pop operations

main.cpp Share Run Output

```cpp
#include <iostream>
using namespace std;

template <class T>
class Stack {
    T arr[100];
    int top;

public:
    Stack() {
        top = -1;
    }

    void push(T value) {
        if (top == 99)
            cout << "Stack Overflow" << endl;
        else
            arr[++top] = value;
    }

    void pop() {
        if (top == -1)
            cout << "Stack Underflow" << endl;
        else
            cout << "Popped element: " << arr[top--] << endl;
    }

    void display() {
        if (top == -1)
            cout << "Stack is empty" << endl;
        else {
            for (int i = top; i >= 0; i--)
                cout << arr[i] << " ";
            cout << endl;
        }
    }
};

int main() {
    Stack<int> s;

    s.push(10);
    s.push(20);
    s.push(30);

    s.display();
    s.pop();
    s.display();

    return 0;
}
```

```
30 20 10
Popped element: 30
20 10


=== Code Execution Successful ===
```

---

2. Queue with enqueue and dequeue operations

```cpp
#include <iostream>
using namespace std;

template <class T>
class Queue {
    T arr[100];
    int front, rear;

public:
    Queue() {
        front = -1;
        rear = -1;
    }

    void enqueue(T value) {
        if (rear == 99)
            cout << "Queue Overflow" << endl;
        else {
            if (front == -1)
                front = 0;
            arr[++rear] = value;
        }
    }

    void dequeue() {
        if (front == -1 || front > rear)
            cout << "Queue Underflow" << endl;
        else
            cout << "Dequeued element: " << arr[front++] <<
                endl;
    }

    void display() {
        if (front == -1 || front > rear)
            cout << "Queue is empty" << endl;
        else {
            for (int i = front; i <= rear; i++)
                cout << arr[i] << " ";
            cout << endl;
        }
    }
};

int main() {
    Queue<int> q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();
    q.dequeue();
    q.display();

    return 0;
}
```

```
10 20 30
Dequeued element: 10
20 30


=== Code Execution Successful ===
```

3. Store and display a pair of values

```cpp
#include <iostream>
using namespace std;

template <class T, class U>
class Pair {
    T first;
    U second;

public:
    Pair(T a, U b) {
        first = a;
        second = b;
    }

    void display() {
        cout << "First value: " << first << endl;
        cout << "Second value: " << second << endl;
    }
};

int main() {
    Pair<int, double> p(10, 5.5);

    p.display();

    return 0;
}
```

```
First value: 10
Second value: 5.5

=== Code Execution Successful ===
```

---

4. Basic arithmetic operations

```cpp
#include <iostream>
using namespace std;

template <class T>
class Arithmetic {
    T a, b;

public:
    Arithmetic(T x, T y) {
        a = x;
        b = y;
    }

    void display() {
        cout << "Addition: " << a + b << endl;
        cout << "Subtraction: " << a - b << endl;
        cout << "Multiplication: " << a * b << endl;

        if (b != 0)
            cout << "Division: " << a / b << endl;
        else
            cout << "Division not possible" << endl;
    }
};

int main() {
    Arithmetic<double> obj(10.5, 2.5);

    obj.display();

    return 0;
}
```

```
First value: 10
Second value: 5.5

=== Code Execution Successful ===
```

---

5. Input and display array elements

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;

template <class T>
class Array {
    T arr[100];
    int n;

public:
    void input() {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter elements: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }

    void display() {
        cout << "Array elements are: ";
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
    }
};

int main() {
    Array<int> a;

    a.input();
    a.display();

    return 0;
}
```

Output Clear

```
Enter number of elements: 5
Enter elements: 1 2 3 4 5
Array elements are: 1 2 3 4 5

=== Code Execution Successful ===
```